# Molfig

## Static molecular structure rendering for Typst

v0.1.1 2026-06-19 MIT

Render molecular structures through maquette.

Eito Yoneyama

Molfig turns PDB, mmCIF, and BinaryCIF structures into static OBJ, STL, or PLY meshes inside a Rust WebAssembly plugin, then hands those meshes to maquette for document rendering.

## Table of Contents

# Part I

# Overview

Molfig is a Typst package for molecular figures in static documents. It accepts PDB, mmCIF, and BinaryCIF input, converts the structure into a CPU-side Mol*-style Model/Structure/Unit representation, builds static geometry, exports the mesh as OBJ, STL, or PLY, and delegates final rendering to maquette.

Mol* is an open-source molecular visualization toolkit and web viewer. Molfig uses Mol* as the compatibility target for structure interpretation, static geometry, and OBJ/STL export behavior; see molstar.org[1] and molstar/molstar[2].

## I.1 Design Goals

Molfig is designed around four constraints:

1. Typst packages cannot invent paths into the caller's project. On Typst 0.15.0 or later, the caller may pass a project-side `path(...)` value and Molfig will read it as bytes. For Typst 0.14-compatible documents, the caller should pass bytes produced by `read(..., encoding: none)`.
2. The plugin should run in Typst's WebAssembly environment without WebGL.
3. The molecular path should preserve Mol*-style structure concepts rather than flattening everything at parse time.
4. The final image should be a document-native static rendering through maquette, not an interactive viewer snapshot.

> Molfig is not a Mol* WebGL renderer. It intentionally excludes interactive WebGL surface and volume visuals such as molecular surface, gaussian surface, gaussian volume, and density maps. Its public contract is structure parsing, static mesh generation, OBJ/STL/PLY export, and maquette rendering.

## I.2 Rendering Pipeline

| Stage | Control | Role |
|---|---|---|
| **Read** | `path(...)` or `read(..., encoding: none)` | The user document supplies either a Typst 0.15+ project path or already-read structure bytes. |
| **Parse** | `format` | The plugin parses PDB, text CIF/mmCIF, or BinaryCIF MessagePack data. |
| **Model** | `assembly`, `alt-loc` | Model, Structure, Unit, unit operators, altLoc policy, bonds, secondary structure, and coarse data are selected. |

[1] https://molstar.org/
[2] https://github.com/molstar/molstar

| | | |
|---|---|---|
| **Geometry** | `representation`, `quality` | The static geometry layer builds spheres, cylinders, tubes, sheets, ribbons, nucleotide visuals, carbohydrate visuals, and related mesh spans. |
| **Export** | `mesh-format` | The mesh is serialized as OBJ, binary STL, or ASCII PLY bytes. |
| **Render** | `config` | The exported bytes and maquette configuration are passed to maquette. |

# Part II

# Quickstart

The following example renders PDB entry 9R1O. Put `9R1O.typ` and `9R1O.pdb` in the same directory, then compile the Typst file.

Complete 9R1O example — *9R1O.typ*

```
#import "@preview/molfig:0.1.1"

#set page(width: auto, height: auto, margin: 0mm)

// Uses structural data from RCSB PDB / wwPDB.
// PDB ID: 9R1O
// PDB DOI: https://doi.org/10.2210/pdb9R1O/pdb
// PDB archive data files are available under CC0 1.0.
#let pdb = path("9R1O.pdb")

#molfig.render(
  pdb,
  format: "pdb",
  representation: "molstar",
  assembly: "1",
  mesh-format: "obj",
  quality: "high",
  center: true,
  output-format: "png",
  config: (
    azimuth: 35,
    elevation: 24,
    background: "",
  ),
)
```

The `path("9R1O.pdb")` input in this example requires Typst 0.15.0 or later. On an older Typst version, replace it with `read("9R1O.pdb", encoding: none)` and pass the resulting bytes to `molfig.render`.

Compile the example first when you want to refresh the figure embedded below:

```
typst compile 9R1O.typ
```

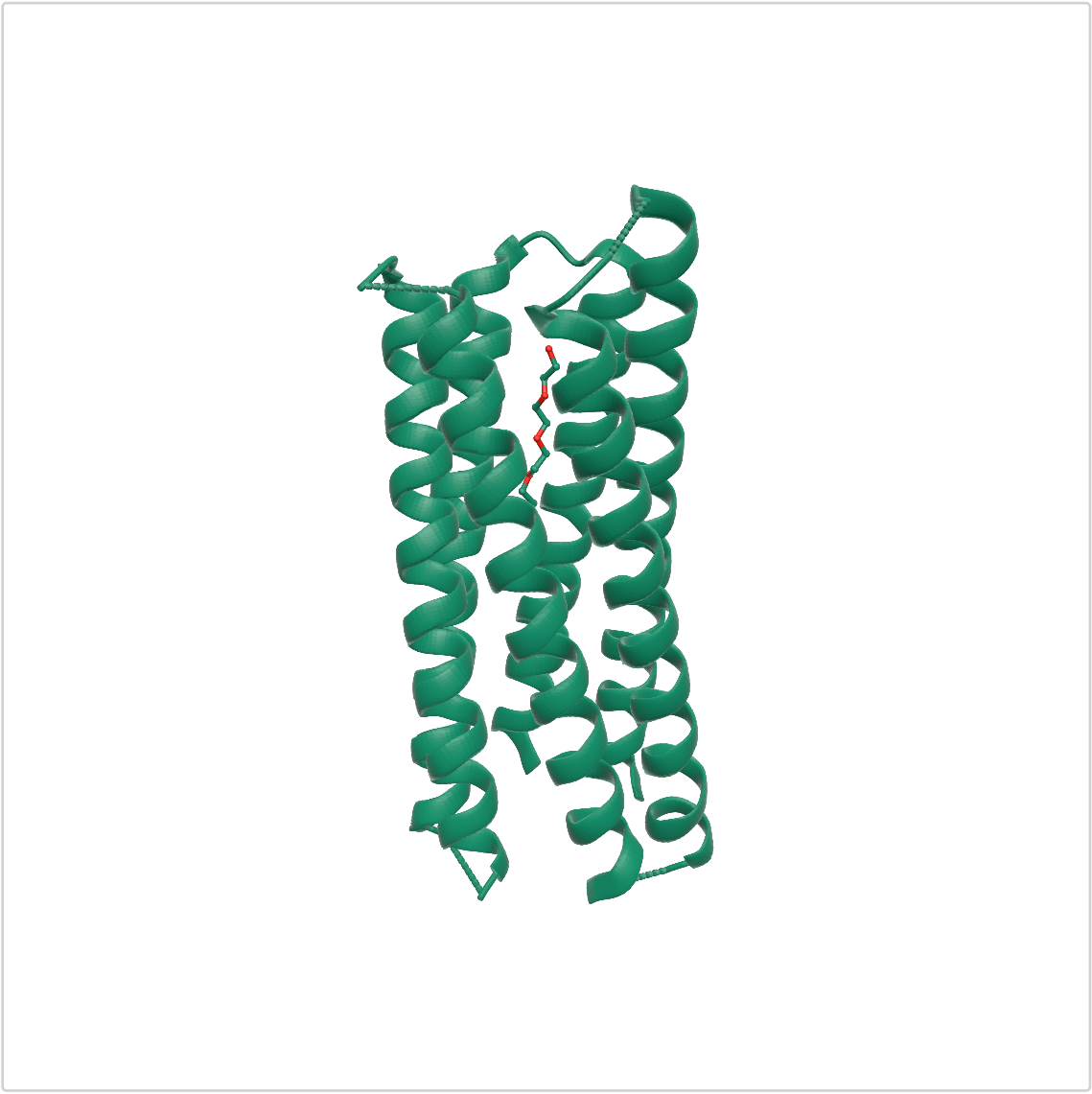

Figure 1: 9R1O rendered by Molfig from PDB entry 9R1O.

Structural data source: RCSB PDB / wwPDB, PDB ID `9R1O`, DOI `10.2210/pdb9R1O/pdb`. PDB archive data files are distributed under CC0 1.0.

## II.1 Reading Structure Files

On Typst 0.15.0 or later, prefer passing a `path(...)` value for file-backed structure data. The path is resolved by Typst relative to the caller's file, and Molfig reads it internally with `encoding: none`. This keeps package calls concise while preserving the caller-side project boundary.

Typst 0.15+ path inputs

```
#let pdb = path("entry.pdb")
#let cif = path("entry.cif")
#let bcif = path("entry.bcif")
```

For documents that must also compile on Typst 0.14, read the file in the caller document and pass the resulting bytes. Always use `encoding: none` for external structure files; it preserves BinaryCIF bytes and avoids Unicode decoding loss for fixed-width PDB columns.

Typst 0.14-compatible byte inputs

```
#let pdb = read("entry.pdb", encoding: none)
#let cif = read("entry.cif", encoding: none)
#let bcif = read("entry.bcif", encoding: none)
```

Passing a Typst string is accepted for small inline examples. A string is treated as inline molecular text, not as a file path.

## II.2 Choosing The First Options

| Question | Recommended setting | Why |
|---|---|---|
| I want the closest default molecular figure. | `representation: "molstar"` | Uses the package's Mol*-style static visual set. |
| I want a protein cartoon. | `representation: "cartoon"` | Uses polymer trace, secondary structure, nucleotide, and gap visuals. |
| I need readable geometry diffs. | `mesh-format: "obj"` | OBJ is text and preserves face groups. |
| I need compact triangle bytes. | `mesh-format: "stl"` | STL is binary and simple for downstream mesh tools. |
| I want text mesh plus group metadata. | `mesh-format: "ply"` | PLY is ASCII and carries package-owned group comments/properties. |
| Compilation is slow. | `quality: "auto"` | Lets Molfig pick lower tessellation for large structures. |

# Part III

# Public API

Every public command takes `(data)` as its first positional argument. It can be bytes from `read(..., encoding: none)`, inline molecular text as a string, or a Typst 0.15+ `path(...)` value. The same mesh options are shared by render, render-object, export, and metadata commands unless noted otherwise.

| Command | Returns | Use when |
|---|---|---|
| `#render` | content | You want the figure directly in the document. |
| `#render-object` | dictionary | You want the rendered content plus raw mesh bytes and metadata. |
| `#to-obj` | bytes | You need OBJ mesh bytes for maquette or an external tool. |
| `#to-mtl` | bytes | You need the companion material library for OBJ output. |
| `#to-stl` | bytes | You need binary STL triangle data. |
| `#to-ply` | bytes | You need ASCII PLY output with Molfig metadata comments/properties. |
| `#info` | dictionary | You want structure and render planning metadata without rendering. |
| `#mesh-info` | dictionary | You want maquette's mesh metadata for the generated OBJ/STL/PLY bytes. |

## III.1 Rendering Commands

```
#render(
  (data),
  (format),
  (mesh-format),
  (representation),
  (config),
  (width),
  (height),
  (output-format)
)
```

Converts molecular bytes to a static mesh and delegates rendering to maquette.

Argument

`(data)` — `bytes` | `str`

Molecular structure input. Pass `path("file.pdb")` on Typst 0.15+, or `read("file.pdb", encoding: none)` for Typst 0.14-compatible documents.

Argument

`(format): "auto"` `str`

Input parser selection. Use an explicit format for reproducible documents.

Argument

`(mesh-format): "obj"` `str`

Intermediate mesh format sent to maquette.

Argument

`(config): (:)` `dictionary`

JSON-encoded and passed to maquette. For OBJ rendering, Molfig adds the active color theme as a maquette material map; explicit `config.materials` entries override generated colors with the same material id.

Argument

`(width): "auto"` `str`

Width forwarded to the maquette rendering command.

Argument

`(height): "auto"` `str`

Height forwarded to the maquette rendering command.

Argument

`(output-format): "png"` `str`

Output format forwarded to maquette. PNG is the safest default for complex meshes.

**`#render-object((data), (mesh-format))`**

Returns a dictionary with:

| Key | Value |
|---|---|
| `kind` | The string `"render-object"`. |
| `format` | The selected mesh format. |
| `mesh_format` | The selected mesh format with snake_case naming. |
| `mesh` | Generated OBJ/STL/PLY bytes. |
| `materials` | Generated maquette material map for OBJ, or an empty dictionary for STL/PLY. |
| `info` | The same dictionary returned by `#info` for the same mesh options. |

| | |
|---|---|
| `content` | The Typst content returned by `#render`. |

Render and inspect in one call

```
#let object = molfig.render-object(
  data,
  format: "mmcif",
  representation: "cartoon",
  mesh-format: "ply",
  assembly: "1",
  config: (azimuth: 30, elevation: 18, background: ""),
  width: 54mm,
)

#object.content

#metadata((
  mesh-format: object.mesh_format,
  atom-count: object.info.atom_count,
))
```

## III.2 Export Commands

**`#to-obj((data))`**
Returns OBJ bytes. OBJ is the most useful interchange format when a downstream tool wants text geometry, material references, and group labels.

**`#to-mtl((data))`**
Returns the companion MTL material library for OBJ output. Use it alongside `#to-obj` when exporting outside Typst.

**`#to-stl((data))`**
Returns binary STL bytes. STL follows Mol* static exporter behavior: the header starts with `"Exported from Mol*"` and the two-byte facet attribute field is kept at zero.

**`#to-ply((data))`**
Returns ASCII PLY bytes. PLY is a Molfig-owned static mesh format because the pinned Mol* geo-export extension does not provide PLY output.

Direct mesh export

```
#let data = read("structure.bcif", encoding: none)

#let obj = molfig.to-obj(data, format: "bcif", representation: "molstar")
#let mtl = molfig.to-mtl(data, format: "bcif", representation: "molstar")
#let stl = molfig.to-stl(data, format: "bcif", representation: "molstar")
#let ply = molfig.to-ply(data, format: "bcif", representation: "molstar")
```

## III.3 Metadata Commands

**`#info(data)`**

Parses the structure and returns molecular and mesh-planning metadata without maquette rendering. This is the fastest public command for debugging parser, assembly, altLoc, bond, secondary structure, and representation behavior.

**`#mesh-info(data, mesh-format)`**

Generates a mesh, then delegates mesh metadata extraction to maquette's `get-obj-info`, `get-stl-info`, or `get-ply-info` helper.

# Part IV

# Shared Mesh Options

The following options are accepted by `#render`, `#render-object`, `#to-obj`, `#to-mtl`, `#to-stl`, `#to-ply`, and `#info`.

## IV.1 Input Selection

| Option | Default | Meaning |
|---|---|---|
| `⟨format⟩` | `"auto"` | Input parser. Supported values: `"auto"`, `"pdb"`, `"cif"`, `"mmcif"`, `"bcif"`, `"binarycif"`, `"binary-cif"`. |
| `⟨block-index⟩` | `none` | Zero-based BinaryCIF data block index. Useful when one BinaryCIF file contains multiple data blocks. |
| `⟨block-header⟩` | `""` | Exact BinaryCIF block header. When set, it takes precedence over `⟨block-index⟩`. |

| Format | Typical extension | Notes |
|---|---|---|
| `"pdb"` | `.pdb` | Fixed-width text. Keep bytes to preserve columns. |
| `"cif"` or `"mmcif"` | `.cif`, `.mmcif` | Text CIF categories and loops. |
| `"bcif"` | `.bcif` | BinaryCIF MessagePack data with encoded columns and optional masks. |
| `"auto"` | any | Convenient for exploration, but explicit formats are better for release documents. |

BinaryCIF numeric atom-site columns are decoded into typed accessors before falling back to string cells. Molfig does not round-trip BinaryCIF through synthetic CIF text.

## IV.2 Structure Selection

| Option | Default | Meaning |
|---|---|---|
| `⟨assembly⟩` | `"1"` | Biological assembly id. Use `"asymmetric-unit"`, `"none"`, or an empty string for source coordinates without assembly operators. |
| `⟨alt-loc⟩` | `""` | Alternate location id or policy. Supports concrete ids such as `"A"`, plus `"highest-occupancy"` and `"all"`. |

| | | |
|---|---|---|
| `⟨infer-bonds⟩` | `true` | Infer covalent bonds when explicit bond data are absent. |
| `⟨center⟩` | `true` | Recenters exported vertices around Mol*-style visible export bounds. |

Assembly expansion keeps the source model and unit operators as the semantic structure boundary, then materializes mesh coordinates for export. This means metadata can still report selected assembly operators even though OBJ/STL/PLY are static mesh formats.

# IV.3 Representation Selection

| Option | Default | Meaning |
|---|---|---|
| `⟨representation⟩` | `"molstar"` | Static Mol*-style default visual set. Also accepts `"default"` and `"auto"`. |
| `⟨color-theme⟩` | `"chain-id"` | Assigns Mol* Chain ID colors to polymer chains and preserves element-symbol colors used by atomic visuals. `"chain-id"` is the currently supported theme. Color is serialized through OBJ material references and the companion MTL output. STL and the current PLY output do not carry these colors, so themed document rendering requires `mesh-format: "obj"`. |

## IV.3.1 Chain ID Color Rules

The `"chain-id"` theme follows these rules:

- For atomic models, the author chain id (`auth_asym_id`) is used when it is present; otherwise the label chain id (`label_asym_id`) is used. PDB input normally has the same value for both. Coarse IHM spheres and Gaussians use their asym id.
- Unique chain ids are collected in model order. Atomic chains are collected first, followed by previously unseen coarse-sphere and coarse-Gaussian chain ids. Assembly copies retain the source chain id and therefore retain its color.
- Colors are assigned from the following 25-color Mol* many-distinct palette. If a structure contains more than 25 chain ids, the palette repeats from the first color.

| Palette positions | Colors |
|---|---|
| 1–5 | `#1b9e77` , `#d95f02` , `#7570b3` , `#e7298a` , `#66a61e` |
| 6–10 | `#e6ab02` , `#a6761d` , `#666666` , `#e41a1c` , `#377eb8` |
| 11–15 | `#4daf4a` , `#984ea3` , `#ff7f00` , `#ffff33` , `#a65628` |
| 16–20 | `#f781bf` , `#999999` , `#66c2a5` , `#fc8d62` , `#8da0cb` |
| 21–25 | `#e78ac3` , `#a6d854` , `#ffd92f` , `#e5c494` , `#b3b3b3` |

Chain-associated geometry, including cartoon, ribbon, backbone, nucleotide, carbohydrate, gap, and coarse visuals, uses the assigned chain color unless the visual has an explicit atomic material. Atomic visuals apply a chain-and-element rule:

- Carbon uses its chain color for ordinary atoms, polymers, ligands, and branched components.
- Water, ion, and lipid components use element-symbol colors for every atom, including carbon.
- Non-carbon atoms use their element-symbol color. The symbol is read from `type_symbol` when available, otherwise from the parsed element field. Unknown symbols fall back to white.
- Bond geometry inherits the material of its source atom. Component visuals that emit two directed half-bonds consequently color each half from its adjacent atom.

Element-symbol colors include Mol*'s default lightness adjustment. The final RGB values written to OBJ materials are:

| Elements | Final colors |
|---|---|
| Hydrogen isotopes | H `#ffffff` , D `#ffffca` , T `#ffffaa` |
| Common organic elements | C `#999999` , N `#4259ff` , O `#ff2618` , P `#ff8a14` , S `#ffff3e` , Se `#ffab17` |
| Halogens | F `#9aea5a` , Cl `#37fb2e` , Br `#b13431` , I `#9e179e` |
| Alkali and alkaline-earth metals | Na `#b566fd` , Mg `#95ff1f` , K `#994ade` , Ca `#4eff1e` |
| Transition metals | Mn `#a683d1` , Fe `#eb703c` , Co `#fb9aaa` , Ni `#5cda5a` , Cu `#d3893c` , Zn `#8689ba` |
| Other or unknown | `#ffffff` |

The companion MTL records opacity `0.3` for branched components, `0.6` for water and lipids, and `1.0` otherwise. Molfig's automatic maquette material map currently forwards RGB colors only; it does not forward these MTL opacity values.

| Value | Best for | Main geometry | Notes |
|---|---|---|---|
| `"molstar"` | General figures | Polymer traces, element spheres, bonds, carbohydrates, nucleotides, gaps | Closest default for static Molfig output. |
| `"spacefill"` | Atomic packing | Atom spheres | Uses atom radii and `(radius-scale)`. |
| `"ball-and-stick"` | Ligands and small molecules | Atom spheres plus bond cylinders | Good for local chemistry and explicit bond inspection. |
| `"cartoon"` | Proteins and nucleic acids | Polymer trace tubes/sheets/ribbons, nucleotide visuals, gaps | Uses secondary structure and polymer trace metadata. |

| | | | |
|---|---|---|---|
| `"ribbon"` | Backbone shape | Ribbon-oriented polymer geometry | Good for broad fold visibility. |
| `"backbone"` | Trace-only views | Backbone cylinders and spheres | Lower-detail alternative for polymer paths. |

## IV.4 Geometry And Quality

| Option | Default | Meaning |
|---|---|---|
| `(quality)` | `"custom"` | `"custom"` preserves explicit numeric controls. `"auto"` chooses a preset from structure size. Also accepts `"highest"`, `"higher"`, `"high"`, `"medium"`, `"low"`, `"lower"`, `"lowest"`. |
| `(sphere-detail)` | `2` | Sphere tessellation detail. Public input is clamped by the plugin. |
| `(linear-segments)` | `8` | Curve subdivisions for polymer and tube paths. |
| `(radial-segments)` | `16` | Radial subdivisions for cylinders, tubes, and ribbon-like profiles. |
| `(radius-scale)` | `1.0` | Global radius multiplier. |
| `(atom-radius)` | `0.28` | Explicit atom radius for simple representations. |
| `(bond-radius)` | `0.12` | Explicit bond radius for cylinder-based bond visuals. |
| `(ribbon-radius)` | `0.2` | Tube/ribbon radius control. |
| `(ribbon-width)` | `0.55` | Ribbon width control. |
| `(helix-profile)` | `"elliptical"` | `"elliptical"`, `"rounded"`, or `"square"`. `"sheet"` is accepted as an alias for square. |
| `(round-cap)` | `false` | Enable rounded caps where supported by the selected cartoon/ribbon path. |
| `(sheet-arrow-factor)` | `1.5` | Scales sheet arrow geometry. |
| `(tubular-helices)` | `false` | Prefer tubular helix geometry in supported cartoon paths. |

Quality presets override the explicit numeric tessellation controls. Use `quality: "custom"` when exact reproducibility matters, and use `quality: "auto"` or a lower preset when large structures compile too slowly.

| Preset | Sphere detail | Radial segments | Linear segments |
|---|---|---|---|
| `"highest"` | 3 | 36 | 18 |

| | | | |
|---|---|---|---|
| `"higher"` | 3 | 28 | 14 |
| `"high"` | 2 | 20 | 10 |
| `"medium"` | 1 | 12 | 8 |
| `"low"` | 0 | 8 | 3 |
| `"lower"` | 0 | 4 | 2 |
| `"lowest"` | 0 | 2 | 1 |

> The public default is `quality: "custom"`, so the numeric defaults in this manual are honored unless you explicitly choose a preset or `"auto"`.

# Part V

# Maquette Rendering

`#render` calls one of maquette's mesh renderers based on `(mesh-format)`: `render-obj`, `render-stl`, or `render-ply`. The `(config)` dictionary is JSON-encoded and passed through. For OBJ, Molfig derives maquette's `materials` dictionary from the exported material ids so `color-theme: "chain-id"` is visible in the document. User-supplied material entries are merged last and therefore take precedence. STL has no material channel, and Molfig's current PLY schema contains no vertex or face color properties. Consequently, maquette renders STL and PLY without the selected color theme.

Camera and background passthrough

```
#let cif = read("structure.cif", encoding: none)

#molfig.render(
  cif,
  format: "cif",
  representation: "ball-and-stick",
  mesh-format: "ply",
  config: (
    azimuth: 45,
    elevation: 20,
    background: "",
  ),
  width: 75mm,
)
```

Molfig does not validate maquette-specific keys. Unknown keys are passed to maquette, so maquette remains the source of truth for camera and renderer settings.

## V.1 Choosing A Mesh Format

| Format | Command | Strength | Tradeoff |
|---|---|---|---|
| OBJ | `#to-obj` | Readable text, group labels, material references | Larger than binary STL and needs MTL when material rows are consumed externally. |
| MTL | `#to-mtl` | Companion material rows for OBJ | Not a geometry format by itself. |
| STL | `#to-stl` | Compact binary triangle stream | No stable text metadata channel; facet attribute bytes are zero. |

| | | | |
|---|---|---|---|
| PLY | `#to-ply` | Text mesh with Molfig group metadata | Package-owned output, not a pinned Mol* exporter format. |

For visual documents, start with OBJ. Switch to STL only when the downstream consumer specifically wants binary STL, and use PLY when the group metadata is more useful than OBJ/MTL compatibility.

# Part VI

# Metadata Reference

`#info` returns a dictionary. It is intended for document logic, regression tests, and debugging. The exact set of keys may grow as Molfig's Mol*-parity layer grows, but the current major groups are:

| Key | Type | Meaning |
|---|---|---|
| `atom_count` | integer | Visible atom count after selected assembly/altLoc handling and geometry expansion. |
| `bond_count` | integer | Visible bond count used by static geometry. |
| `alt_locs` | array | Available alternate location ids from the source coordinates. |
| `alt_locs_info` | dictionary | Selected altLoc policy and available ids. |
| `assemblies` | array | Available biological assembly summaries. |
| `assembly` | dictionary | Selected assembly id. |
| `source_data` | dictionary | Input source categories, row counts, and original source kind. |
| `structure` | dictionary | Model/Structure/Unit counts, boundary, conformation, segments, ranges, and lookup3d summary. |
| `secondary_structure` | dictionary | Helix and sheet ranges. |
| `representation` | dictionary | Selected and realized visual names. |
| `render_objects` | array | Semantic render-object spans with geometry type, visual, chain, residue range, group id, and value-cell style counts. |
| `bond_metadata` | dictionary | Counts by bond source and flags such as struct_conn, index_pair, chem_comp, aromatic, and resonance. |
| `bounds` | dictionary | Expanded coordinate bounds before export centering. |

Basic 9R1O metadata query

```
#let pdb = read("9R1O.pdb", encoding: none)
#let meta = molfig.info(pdb, format: "pdb", assembly: "1")

Atoms: #meta.atom_count
Bonds: #meta.bond_count
Assembly: #meta.assembly.id
```

Render-object metadata is useful when validating what Molfig actually built:

Representation assertions

```
#let meta = molfig.info(
  data,
  format: "mmcif",
  representation: "cartoon",
  assembly: "1",
  alt-loc: "highest-occupancy",
)

#assert(meta.render_objects.any(object => object.secondary_type == "helix"))
#assert(meta.representation.realized_visuals.contains("polymer-trace"))
```

# VI.1 Assembly And Unit Metadata

Assembly selection is visible through both `info(...).structure` and mesh export metadata. OBJ and PLY include `molfig_operator_metadata` comments when an assembly is selected. STL has no equivalent text channel, so assembly operator metadata should be inspected through `#info` or `#render-object`.

| Path | Example | Meaning |
|---|---|---|
| `structure.unit_count` | `meta.structure.unit_count` | Number of units after structure construction. |
| `structure.symmetry_group_count` | `meta.structure.symmetry_group_count` | Mol*-style symmetry grouping count. |
| `structure.coordinate_system` | `meta.structure.coordinate_system` | Selected coordinate operator summary. |
| `structure.boundary` | `meta.structure.boundary.sphere_radius` | Boundary sphere and box used by structure-level geometry. |
| `structure.lookup3d` | `meta.structure.lookup3d.unit_count` | Lookup3D summary for constructed units. |

# Part VII

# Practical Workflows

## VII.1 A Publication Figure

Pinned 9R1O figure

```
#import "@preview/molfig:0.1.1"

#let pdb = read("9R1O.pdb", encoding: none)

#figure(
  molfig.render(
    pdb,
    format: "pdb",
    representation: "molstar",
    assembly: "1",
    mesh-format: "obj",
    quality: "high",
    center: true,
    config: (
      azimuth: 35,
      elevation: 24,
      background: "",
    ),
    width: 90mm,
  ),
  caption: [9R1O rendered with #product-name.],
)
```

## VII.2 A Lightweight Draft Figure

Large assembly draft

```
#molfig.render(
  read("data/large-assembly.bcif", encoding: none),
  format: "bcif",
  representation: "molstar",
  assembly: "1",
  quality: "auto",
  mesh-format: "stl",
  config: (background: ""),
  width: 60mm,
)
```

## VII.3 Export For External Tools

OBJ and MTL bytes

```
#let data = read("data/ligand.cif", encoding: none)
#let mesh = molfig.to-obj(
  data,
  format: "cif",
  representation: "ball-and-stick",
  assembly: "asymmetric-unit",
  center: false,
)
#let material = molfig.to-mtl(data, format: "cif", representation: "ball-and-stick")
```

Typst does not write arbitrary files from a package call. Use these byte values inside Typst document logic, or expose them through a workflow that is allowed to write artifacts outside Typst.

# Part VIII
# Troubleshooting

| Symptom | Likely cause | Action |
|---|---|---|
| Plugin says Molfig expects bytes. | The input was not bytes, inline string data, or a Typst 0.15+ path value. | Pass `path("entry.pdb")` on Typst 0.15+, or pass `read("entry.pdb", encoding: none)` for Typst 0.14-compatible documents. |
| BinaryCIF reports a missing encoding. | The file is not valid BinaryCIF MessagePack or a column lacks required BinaryCIF encoding metadata. | Check the source file and use `format: "cif"` only for text CIF/mmCIF. |
| The figure is sparse or missing chains. | Assembly or altLoc selection filtered the structure. | Inspect `molfig.info(...).assemblies` and `alt_locs_info`, then set `⟨assembly⟩` and `⟨alt-loc⟩` explicitly. |
| A large assembly compiles slowly. | High tessellation or too much assembly geometry. | Use `quality: "auto"`, lower `⟨radial-segments⟩` and `⟨linear-segments⟩`, or choose a lighter representation. |
| Bonds are missing. | The file lacks explicit bonds and inference is disabled. | Leave `⟨infer-bonds⟩` as `true`, or inspect `bond_metadata` to see available bond sources. |
| Rendered view differs from external OBJ inspection. | Different camera, centering, or mesh format. | Pin `⟨center⟩`, `⟨mesh-format⟩`, maquette `⟨config⟩`, and representation quality options. |

For reproducible documents, avoid implicit choices:

Reproducible 9R1O option block

```
#let pdb = read("9R1O.pdb", encoding: none)

#molfig.render(
  pdb,
  format: "pdb",
  representation: "molstar",
  assembly: "1",
  mesh-format: "obj",
  quality: "custom",
  sphere-detail: 2,
  linear-segments: 8,
  radial-segments: 16,
  center: true,
  config: (azimuth: 35, elevation: 24, background: ""),
)
```

# Part IX

# License And Notices

Molfig package code is distributed under the MIT License. The package also contains third-party material that is covered separately:

| Material | Scope | License / notice |
|---|---|---|
| `Mol*` | `molfig.wasm` contains Rust behavior and generated reference data derived from `Mol*` parity work. | MIT License; copyright (c) 2017 - now, `Mol*` contributors. |
| PDB archive data | `examples/data/*.pdb`, `examples/data/*.cif`, `examples/data/*.bcif`, and generated example PDFs. | CC0 1.0 Universal Public Domain Dedication; attribution to PDB IDs and structure authors is encouraged. |
| Typst helper packages | `maquette` for rendering and `mantys` for this manual. | Imported by Typst; not vendored by Molfig. |

The package includes `NOTICE.md` and `THIRD_PARTY_NOTICES.md` with the full distribution notice. Example data attributions are also listed in `examples/data/README.md` and `examples/data/ATTRIBUTION.tsv`.

For figures made from PDB archive data, cite the PDB ID and DOI. For example:

Suggested data attribution

```
Structural data source: RCSB PDB / wwPDB, PDB ID 9R1O,
https://doi.org/10.2210/pdb9R1O/pdb.
PDB archive data files are available under CC0 1.0.
```

When describing `Mol*` parity or comparing against `Mol*` output, cite `Mol*`:

### Suggested Mol* citation

```
Sehnal et al. Mol* Viewer: modern web app for 3D visualization and
analysis of large biomolecular structures.
Nucleic Acids Research 49:W431-W437 (2021).
https://doi.org/10.1093/nar/gkab314
```

No endorsement by Mol*, RCSB PDB, wwPDB, structure authors, or data providers is implied.

# Part X

# Development

The package consists of a Typst wrapper, a Rust WebAssembly plugin, tests, and documentation:

| Path | Role |
|---|---|
| `package/lib.typ` | Public Typst API and maquette bridge. |
| `wasm-plugin/src/` | Rust parser, Model/Structure/Unit layer, geometry builders, exporters, and Typst plugin ABI. |
| `wasm-plugin/tests/` | Rust and Typst smoke/regression tests. |
| `package/docs/documentation.typ` | This Mantys manual. |
| `package/docs/documentation.pdf` | Compiled reference manual. |
| `package/molfig.wasm` | Checked-in WebAssembly plugin consumed by Typst. |

```
cd wasm-plugin
cargo fmt --check
cargo test
cargo build --release --target wasm32-unknown-unknown
cp target/wasm32-unknown-unknown/release/molfig.wasm ../package/molfig.wasm
cd ..
typst compile --root package package/docs/documentation.typ package/docs/documentation.pdf
```

Regenerate `package/molfig.wasm` after Rust changes that affect the Typst plugin. Regenerate this PDF after documentation or public API changes.

# Part XI

# Index

## I



## M



## R



## T